



① What is a Post Request?

A post request is an HTTP method used to:

- ① send data to the server
- ② create a new resources (user, product, blog post)

Unlike GET (Which just Fetches data),
POST: Has a request body
Send structured information.

Example:

POST /users

```
{  
  "name": "Aman",  
  "age": 20,  
}
```

This tell server: "create a new user with this data".

② Request Body - What is it?

A request body is the data you send with a post request.

It can be:

JSON

Files

Raw text

In APIs, we usually send JSON in the request body.



③ Why is Request Body Needed?

When creating or updating something you need:

- ⊙ user input
- ⊙ Product info
- ⊙ credentials
- ⊙ details

Examples:

```
{  
  "title": "Learn FastAPI",  
  "completed": False  
}
```

④ How FastAPI Get the Request Body?

FastAPI automatically read the request body when you declare a parameter with a model or type.

Example -

```
from fastapi import FastAPI
```

```
app = FastAPI()
```

```
@app.post("/user")
```

```
def create_user(name: str, age: int):
```

```
    return {"name": name,
```

```
            "age": age}
```

This will: Expect body to have {"name": "...", "age": ...}

- ⊙ validate the types
- ⊙ Return that data in response



Fastapi uses type hints to parse and validate data.

- ⑤ Using Pydantic models.
Most real api use model to structure data.
That's where Pydantic comes in.

What is Pydantic:
Library for data validation

Example Model:

```
from pydantic import BaseModel
class User(BaseModel):
    name: str
    age: int
```

Now use it as in an endpoint:

```
@app.post("/user")
def create_user(user: User):
    return user
```

- ⑥ How FastAPI Parse JSON Automatically

If client sends:

POST /users

Content-Type: application/json

```
{
  "name": "Aman",
  "age": 21
}
```




FastAPI will:

1. Convert JSON to Python dict
2. Create User instance
3. Validate that name is string & age is int.
4. Return on process the object

No manual parsing needed.

⑦ Example: Post Request in FastAPI

```
from fastapi import FastAPI
from pydantic import BaseModel
```

```
app = FastAPI()
```

```
class Todo(BaseModel):
```

```
    title: str
```

```
    completed: bool
```

```
@app.post("/todo")
```

```
def create_todo(todo: Todo):
```

```
    return {"message": "created",
```

```
            "todo": todo}
```

client sends:

Post /todo

```
{
  "title": "Study FastAPI",
  "completed": False
}
```

Server returns:

JSON

```
{
  "message": "created",
  "todo": {
    "title": "Study FastAPI",

```




"completed": False

}

}

⑧ What Happen if Data is Wrong?

FastAPI automatically return a Validation error.

Example: missing a required Field

Client Send:

{

"title": "only title"

}

FastAPI responds:

JSON

{

"default": [

{

"loc": ["body", "completed"],

"msg": "Field required"

}

]

}

No error handling code was written -
FastAPI did it for you.

⑨ Why this is useful

FastAPI makes building API fast
because:

→ No Parsing code needed → Built-in

Validation → Auto error response

→ clean return data → work

With OpenAPI Docs automatically.